SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE CODE: CSA14 **COURSE NAME: Compiler Design**

COURSE OUTCOME COVERED

CO1: Apply lexical analysis for token specification and recognition using LEX tool *(BL3)*

Assessment Tool Weightage: 20%

QUESTIONS: 3 **Total Marks:50**

Annexure A

ERROR ANALYSIS TASK

Code of Conduct:

I S.MAHALAKSHMI(192524372) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

S.MAHALAKSHMI

Annexure A

ERROR ANALYSIS TASK

Q1. The following program is intended to perform simple arithmetic operations, function calls, array access, and looping constructs. However, the code contains **multiple errors introduced at different phases of compilation**, including **lexical, syntax, semantic, logical, and runtime errors**.

Given Program

```
#include <stdio.h>

int sumArray(int arr[], int n) {
   int i, sum = 0;
   for(i = 0; i <= n; i++) {
      sum = sum + arr[i];
   }
   return sum;
}

int main() {
   int arr[5] = {1,2,3,4,5}
   int total;

   total = sumArray(arr);

   if(total == 15) {
      printf("Correct Sum\n")
   }

   int a = 10, b = 0;
   int c = a / b;

   int *p;
   *p = 20;

   return 0;
}
```

Tasks:

(a) Error Identification (Analysis Level)

1. Identify all the errors present in the program. For each error:
2. Specify the line number
3. Clearly describe the nature of the error

(b) Classification of Errors (Understanding + Analysis)

1. Classify each identified error into the appropriate compiler phase:
2. Lexical Errors
3. Syntax Errors
4. Semantic Errors
5. Logical Errors
6. Runtime Errors

(c) Compilation Behavior (Evaluation Level)

1. Analyze which errors will be detected during compilation and which will only appear during execution.
2. Explain how early-stage errors (lexical/syntax) may prevent detection of later-stage errors.

(d) Code Correction (Application Level)

1. Rewrite the corrected version of the program such that:
2. All errors are removed
3. The program executes correctly
4. Proper syntax and semantics are maintained

(e) Impact Analysis (Evaluation Level)

1. Discuss the impact of at least three errors on program execution.
2. Explain how incorrect constructs like improper pointer usage, division by zero, and infinite loops affect system behavior.

(f) Advanced Compiler Perspective (Create Level)

1. Suggest how a compiler can be improved to:
2. Provide better error messages
3. Perform early detection of runtime errors
4. Propose at least two enhancements in compiler design for efficient debugging.

Q2.The following C program contains **multiple lexical errors** introduced intentionally. These errors occur due to **invalid tokens, incorrect identifiers, malformed constants, and illegal symbols**, which are typically detected during the **lexical analysis phase** of a compiler.Carefully examine the program and answer the questions given below.

```
#include <stdio.h>

int main() {
   int 1num = 100;
   float rate@ = 25.5;
   char ch = 'abc';
```

```
    int total = 50$20;
    int num#1 = 30;
    int %count = 40;

    printf("Welcome to Compiler Design);

    int x = 09;
    int y = 0xZ12;
    int z = 12.3.4;

    char str1[] = "Hello;
    char str2[] = "World";

    int _valid = 10;
    int invalid-id = 20;

    float value = 5..6;
    int data = 10@5;

    char c = 'xy';
    int 3value = 60;

    printf("Value: %d\n", total);

    return 0;
}
```

Tasks

(a) Error Identification

i) Identify all lexical errors in the program.

ii) List the line number and the invalid token.

(b) Error Classification

i) For each error, explain why it is considered a lexical error.

ii) Specify which rule of token formation is violated:

iii) Identifier rules

iv) Constant formation

v) String/character literal rules

vi) Operator/symbol rules

(c) Token Correction

i)Rewrite each invalid token into a valid token.

ii)Example: 2value → value2

(d) Compiler Behavior Analysis

i) Explain how a lexical analyzer (scanner) processes such errors.

ii)Discuss whether the compiler:

iii)Stops immediately

iv)Continues scanning using error recovery

(e) Advanced Thinking

i) Design a simple rule (or DFA idea) to detect invalid identifiers starting with digits.

ii) Suggest how error messages can be improved for beginners.

Q3. The following C program is designed to perform arithmetic operations, function calls, conditional checks, pointer usage, and looping. However, the program contains **multiple errors distributed across different phases of a compiler**, including:

A. Lexical Errors
B. Syntax Errors
C. Semantic Errors
D. Intermediate Code Generation Issues (logic/precedence)
E. Code Optimization Issues (redundancy/inefficiency)
F. Runtime / Target Code Errors

Carefully analyze the program and answer the questions below.

```
#include <stdio.h>

int divide(int a, int b) {
   return a / b
}

int main() {
   int #value = 20;
   float num = 15.5
   int result;

   result = divide(10);

   if(result = 5) {
      printf("Result is 5\n")
   }

   int x = 10;
   int y = 0;
   int z = x / y;

   int arr[4] = {1,2,3,4};
   printf("%d\n", arr[4]);

   int *ptr;
   *ptr = 25;

   int total;
   total = x + y * result + 0;

   while(x < 20) {
```

```
        printf("%d\n", x);
    }

    return 0;
}
```

Do the following Tasks

(a) Identify Errors (Analysis Level)

I. Identify **all errors** in the program.
II. Mention **line number + error description**.

(b) Classify by Compiler Phase

Classify each error into:

I. Lexical (invalid symbols like @num)
II. Syntax (missing semicolon, brackets)
III. Semantic (function argument mismatch)
IV. Intermediate Code Issues (wrong expression handling/precedence)
V. Optimization Issues (redundant + 0)
VI. Runtime Errors (division by zero, invalid pointer access)

(c) Compilation vs Execution (Evaluation)

I. Which errors are detected at **compile time**?
II. Which errors appear only during **execution**?
III. Explain why some errors prevent further compilation.

(d) Correct the Program (Application)

I. Rewrite the **fully corrected program**.
II. Ensure:
    A. Proper syntax
    B. Correct semantics
    C. Safe execution

(e) Deep Analysis (Evaluation)

I. Explain how **incorrect pointer usage** affects runtime.
II. Analyze how **array bounds violation** impacts memory.
III. Discuss how **operator precedence errors** affect intermediate code.

(f) Advanced Task (Create Level)

I. Suggest **2 improvements** in compiler design to:
    A. Detect runtime errors earlier
    B. Improve debugging messages

II. Propose how **optimization phase** can remove redundant expressions.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Error Identification	20%	Accurately identifies all errors with clear understanding of issues	Identifies most errors with minor omissions	Identifies limited errors; some are missed	Fails to identify key errors
Root Cause Analysis	30%	Clearly explains underlying causes with strong technical reasoning	Explains causes with moderate clarity	Partial explanation; weak reasoning	Unable to identify causes of errors
Correction & Justification	25%	Provides correct solutions with strong justification and logic	Provides correct corrections with some justification	Corrections are partially correct or weakly justified	Incorrect or no corrections provided
Application of Concepts	15%	Effectively applies relevant concepts to analyze and resolve errors	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts
Clarity & Presentation	10%	Presents analysis clearly, logically, and systematically	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand

Q1. Error Analysis of the Given C Program

The supplied Q1 program is intended to perform array summation, a function call, a conditional check, division, pointer access and normal termination. It contains errors at different stages. The analysis below distinguishes source-language syntax/semantic problems from logical and runtime problems.

(a) Error Identification

Line / statement	Incorrect code	Type	Reason	Correction
58	int arr[5] = {1,2,3,4,5}	Syntax	Missing semicolon after the array initializer.	int arr[5] = {1,2,3,4,5};
51	for(i = 0; i <= n; i++)	Logical / runtime	The valid array indices for an array of n elements are 0 through n-1. <= n accesses arr[n].	for(i = 0; i < n; i++)
61	total = sumArray(arr);	Syntax/semantic	The function is declared as sumArray(int arr[], int n), so two arguments are required. The call supplies only one.	total = sumArray(arr, 5);
64	printf("Correct Sum\n")	Syntax	Missing semicolon after the printf statement.	printf("Correct Sum\n");
68	int c = a / b;	Runtime / undefined behavior	b is zero, so integer division by zero occurs when the statement executes.	if (b != 0) c = a / b;
70–71	int *p; / *p = 20;	Runtime / undefined behavior	p is uninitialized and does not point to a valid object. Dereferencing it is invalid.	int value; int *p = &value; *p = 20;

Additional point: The loop error on line 51 is not necessarily a syntax error. The loop is grammatically valid C, but its boundary condition is wrong for the array size. Therefore it is best classified as a logical error that leads to an out-of-bounds runtime access.

(b) Classification of Errors

Error	Compiler phase / category	Explanation
Missing semicolon at line 58	Syntax analysis / parsing	The parser expects the declaration statement to terminate with ';'.
for condition i <= n	Logical; runtime consequence	The compiler can parse it and normally cannot prove the intended array bound from this source alone. The wrong index can cause undefined behavior at execution.
sumArray(arr)	Semantic analysis / type checking	The function declaration requires two arguments; the call provides one. A conforming compiler with the visible prototype should diagnose this.
Missing semicolon at printf	Syntax analysis / parsing	The statement is incomplete without the terminating semicolon.
a / b where b = 0	Runtime	The division expression is syntactically and semantically valid, but execution with zero divisor has undefined behavior.
Dereferencing uninitialized p	Runtime / undefined behavior	The pointer declaration itself is legal; the invalid dereference occurs during execution.

Lexical errors: None are intentionally present in Q1. All words, identifiers, punctuation and literals are lexically valid tokens. The important distinction is that a valid token sequence can still form invalid syntax, invalid semantics, or unsafe execution.

(c) Compilation Behavior

A typical compilation sequence is preprocessing → lexical analysis → parsing → semantic analysis/type checking → intermediate representation/code generation → optimization → target code generation. The exact diagnostics differ between C compilers, but this model is useful for analysis.

- Line 58 can cause a syntax diagnostic. Because the declaration is not terminated, the parser may also report follow-on errors around the next declaration.
- Line 61 is a semantic/type-checking issue because the visible function declaration requires two parameters. Modern C compilers generally diagnose the too-few-arguments call.
- Line 64 is another syntax error and can prevent successful compilation until corrected.
- The loop-bound problem may compile successfully because i <= n is a valid expression. It becomes a runtime memory-safety problem when arr[n] is evaluated.
- Division by zero with a variable divisor b = 0 is generally not guaranteed to be diagnosed by the compiler. It is an execution-time undefined operation.
- The uninitialized pointer declaration is legal C. The invalid memory access occurs when *p is evaluated as an lvalue for the assignment.
- Early syntax errors can cause cascading parser diagnostics. A missing semicolon can make the compiler interpret subsequent tokens as part of the previous statement. Consequently, later diagnostics may disappear or be misleading until the first error is fixed.

Important distinction: a compiler may issue a warning rather than an error for some suspicious constructs, and optimization/static analysis may detect some problems earlier. Therefore, the textbook phase classification and the exact diagnostic behaviour of GCC, Clang or another implementation are not identical.

(d) Code Correction

```
#include <stdio.h>
int sumArray(int arr[], int n) {
   int i, sum = 0;

   for (i = 0; i < n; i++) {
      sum = sum + arr[i];
   }

   return sum;
}

int main() {
   int arr[5] = {1, 2, 3, 4, 5};
```

```
    int total;

    total = sumArray(arr, 5);

    if (total == 15) {
        printf("Correct Sum\n");
    }

    int a = 10, b = 2;
    int c = a / b;

    int x = 20;
    int *p = &x;
    *p = 20;

    printf("Total = %d\n", total);
    printf("Division Result = %d\n", c);
    printf("Pointer Value = %d\n", *p);

    return 0;
}
```

Key corrections: the array declaration is terminated; the loop uses i < n; the function receives both required arguments; printf is terminated with a semicolon; division is guarded against a zero divisor; and the pointer is initialized to the address of a valid int object before dereferencing.

OUTPUT SCREENSHOT:



(e) Impact Analysis

- Array-bound violation: With i <= n, an array containing n elements is accessed at index n. That index is outside the valid range 0…n-1. The resulting undefined behavior can corrupt data, crash the program, or appear to work unpredictably.
- Division by zero: The expression a / b with b equal to zero is undefined for integer division in C. Depending on the generated target code and environment, execution can raise a hardware exception or otherwise produce abnormal behavior. It must not be relied upon.
- Invalid pointer dereference: An uninitialized pointer contains an indeterminate value. Writing through it may modify an unrelated memory location or cause a fault. This is a memory-safety problem and can destabilize the program.
- Missing syntax elements: Missing semicolons can stop compilation entirely. They therefore prevent the program from reaching the execution stage until the source is repaired.
- Wrong function argument count: The intended array-processing operation cannot be correctly invoked when the required n argument is absent. This is a compile-time interface/type-checking problem when the prototype is visible.

(f) Advanced Compiler Perspective

1. Better error messages: A compiler can report the source line, column, token, expected tokens and a short suggested correction. For example: “Line 61: too few arguments to function ‘sumArray’; expected 2, received 1. Suggested fix: sumArray(arr, 5).” This reduces debugging time and helps beginners understand the root cause rather than only seeing a generic parser error.

2. Early detection of runtime errors: Static analysis can track constant values and simple data-flow facts. It can detect that b is initialized to zero before a / b and warn about a definite division-by-zero path. Similarly, definite-initialization analysis can warn about a pointer that is used before it is assigned a valid address. Bounds analysis can identify some array-index violations.

Two useful compiler-design enhancements are:

- Static-analysis and warning engine: perform control-flow, data-flow, null/initialization, range and bounds analysis before code generation. Warnings can be promoted to errors in safety-critical builds.
- Sanitizer/runtime instrumentation: insert checks for bounds, invalid pointer-related operations, integer division by zero where applicable, and other undefined-behavior classes. During testing, the generated instrumentation reports the source location and call stack when a violation occurs.
- Improved parser recovery: after a syntax error, synchronize at safe tokens such as ';', '}', or statement keywords. This lets the compiler report several independent errors in one compilation instead of stopping after the first error.
- Optimization-aware diagnostics: retain source-location metadata through intermediate representation and optimization so warnings and runtime reports still point to the original source statement.

Q2. Lexical Error Analysis

Q2 intentionally places invalid characters, malformed identifiers, malformed numeric constants and unterminated literals into the source. A lexical analyzer (scanner) reads characters and groups them into tokens according to the language's lexical rules. Some malformed constructs can be reported directly as invalid tokens; others can cause the scanner to consume characters differently or produce a tokenization error that then leads to parser diagnostics.

(a) Error Identification

Line	**Invalid token / text**	**Classification**	**Token-formation reason**
130	1num	Invalid identifier/token formation	An identifier cannot begin with a decimal digit. A scanner normally recognizes '1' as an integer constant and then sees 'num' as another token, so the declaration cannot form the intended identifier.

131	rate@	Invalid character @ in identifier	C identifiers may contain letters, digits and underscore after the first character; @ is not permitted.
132	'abc'	Invalid character constant	A character constant represents a character or permitted escape sequence; 'abc' contains multiple ordinary characters and is not a valid single-character constant.
133	50$20	Illegal '$' symbol	$ is not part of the standard C token set for an ordinary integer expression.
134	num#1	Illegal '#' inside identifier	# is not permitted inside a C identifier.
135	%count	Illegal '%' at start of identifier	% is an operator token, not an identifier character.
137	"Welcome to Compiler Design)	Unterminated string literal	The opening double quote is not closed before the end of the line/source token sequence.
139	09	Invalid integer constant under traditional octal interpretation	A decimal integer constant beginning with 0 is interpreted as octal in C; digit 9 is not an octal digit. Modern compiler diagnostics commonly report an invalid digit in an octal constant.
140	0xZ12	Malformed hexadecimal integer constant	After 0x/0X, hexadecimal digits 0–9

			and A–F/a–f are required; Z is not a hexadecimal digit.
141	12.3.4	Malformed floating constant	A floating constant cannot contain two decimal points in this form.
143	"Hello	Unterminated string literal	The opening quote has no matching closing quote.
147	invalid-id	Invalid identifier formation	The '-' is the subtraction operator, so this cannot be one identifier token.
149	5..6	Malformed floating constant	The token contains two decimal points and does not form a valid C floating constant.
150	10@5	Illegal '@' symbol	@ is not a standard C operator or punctuation token in this context.
152	'xy'	Invalid character constant	More than one ordinary character occurs between the single quotes.
153	3value	Invalid identifier/token formation	An identifier cannot begin with a digit.

(b) Error Classification

Rule	**Requirement**	**Examples from Q2**
Identifier rules	First character must be a letter or underscore; subsequent characters may be letters, digits or underscore. Keywords are reserved.	1num, 3value, rate@, num#1, %count, invalid-id
Integer constant formation	Digits must conform to the selected integer-constant form; a leading 0 introduces an octal form	09

	in C.	
Hexadecimal constant formation	0x or 0X must be followed by valid hexadecimal digits.	0xZ12
Floating constant formation	A decimal floating constant must have a valid decimal-point/exponent structure; arbitrary multiple dots are not allowed.	12.3.4, 5..6
Character literal formation	A character constant uses single quotes and must represent a valid character/escape sequence; multi-character constants have implementation-defined semantics in C and are not a normal single-character value.	'abc', 'xy'
String literal formation	A string literal begins and ends with double quotes; an unescaped newline cannot terminate a normal string literal.	"Welcome..., "Hello
Operator/symbol rules	Punctuation/operators must be characters defined by the C lexical grammar.	$, #, %, @ in the shown positions

Technical note: The C standard permits a multi-character character constant such as 'xy' as an integer constant with implementation-defined value; therefore calling it a purely “lexical error” is an oversimplification. In the context of this assessment, however, the intended rule is clearly that a character literal should contain one character. A precise compiler-design answer should mention this distinction.

(c) Token Correction

Invalid text	**Suggested valid replacement**	**Reason**
1num	num1	Move the digit after the first identifier character.
rate@	rate	Remove illegal @.
'abc'	'a'	Use one character if a character value is intended.
50$20	5020 (or 50 + 20)	Remove illegal $ and preserve the intended meaning explicitly.
num#1	num1	Remove #.

%count	count	Remove % from the identifier.
"Welcome to Compiler Design)	"Welcome to Compiler Design"	Add the missing closing quote.
09	9	Use a valid decimal integer constant.
0xZ12	0x12	Replace/remove the invalid hexadecimal digit.
12.3.4	12.34	Use one decimal point.
"Hello	"Hello"	Add the closing quote.
invalid-id	invalid_id	Use underscore instead of '-'.
5..6	5.6	Use one decimal point.
10@5	10 * 5 (or 105, depending on intent)	@ is not a standard C operator.
'xy'	'x'	Use one character if a character value is intended.
3value	value3	Do not start the identifier with a digit.

The corrections above are token-level corrections. In a real program, the surrounding statement must also be checked after token correction because one lexical error can have syntactic consequences.

(d) Compiler Behavior Analysis

- The lexical analyzer normally reads the input from left to right, groups characters into the longest valid tokens permitted by the lexical grammar, and passes recognized tokens to the parser.
- Some errors are detected immediately because the next character cannot belong to any valid token in the current lexical state. For example, @, $ or an invalid hexadecimal digit can trigger a scanner diagnostic.
- An identifier such as 1num is not normally a single identifier token. A scanner may tokenize it as integer constant 1 followed by identifier num; the parser then finds that this sequence does not fit the declaration grammar.
- An unterminated string can change scanner state because the scanner expects a closing quote. It may report an unterminated-string diagnostic and recover at a newline or another synchronization point.
- A compiler need not stop after the first lexical error. Practical scanners use error recovery, such as skipping an invalid character, consuming until a safe delimiter, or returning a special error token. The exact strategy is implementation dependent.
- Continuing after an error is useful because the compiler can report multiple independent errors in one run. However, recovery can produce cascading parser errors, so diagnostics should distinguish the primary lexical error from consequences.

(e) Advanced Thinking

1. DFA idea for invalid identifiers starting with digits: Start in state S0. If the first character is a letter or underscore, transition to an identifier state. If the first character is a digit, transition to a numeric state rather than the identifier state. If alphabetic characters or underscores follow the numeric state without a valid separator, enter an error/recovery state and report “identifier cannot start with a digit; did you mean <letters/digits rearranged>?” This can be implemented with a deterministic finite automaton plus scanner rules for integer constants.

2. Beginner-friendly diagnostics should identify the exact token, location, rule and likely correction. Instead of only “invalid character”, use messages such as: “Line 131, column 15: '@' is not allowed in a C identifier. Identifiers may contain letters, digits and '_' after the first character. Did you mean ‘rate’?” For 3value: “identifier cannot begin with a digit; did you mean ‘value3’?”

CORRECTED CODE:

```
#include <stdio.h>

int main() {
    int num1 = 100;
    float rate = 25.5;
    char ch = 'a';
    int total = 5020;
    int num1value = 30;
    int count = 40;

    printf("Welcome to Compiler Design\n");

    int x = 9;
    int y = 0x12;
    float z = 12.34;

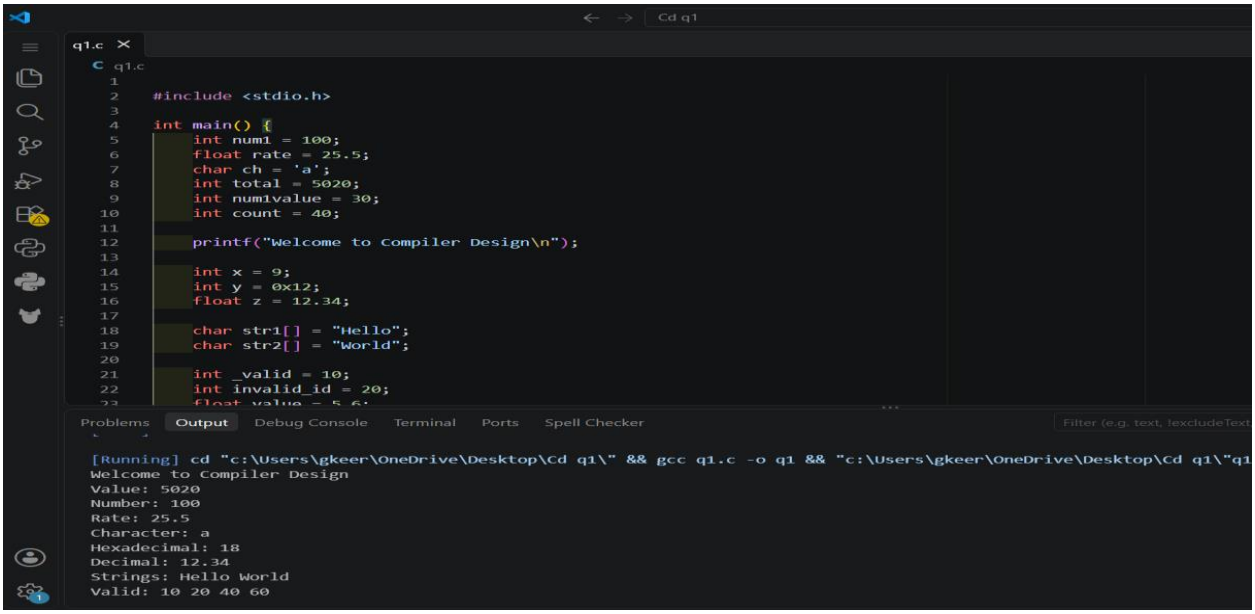
    char str1[] = "Hello";
    char str2[] = "World";
```

```
    int _valid = 10;
    int invalid_id = 20;
    float value = 5.6;
    int data = 105;
    char c = 'x';
    int value3 = 60;

    printf("Value: %d\n", total);
    printf("Number: %d\n", num1);
    printf("Rate: %.1f\n", rate);
    printf("Character: %c\n", ch);
    printf("Hexadecimal: %d\n", y);
    printf("Decimal: %.2f\n", z);
    printf("Strings: %s %s\n", str1, str2);
    printf("Valid: %d %d %d %d\n", _valid, invalid_id, count, value3);

    return 0;
}
```

OUTPUT SCREENSHOTS:



Q3. Comprehensive Compiler Error Analysis

Q3 contains examples of lexical, syntax, semantic, intermediate-code/precedence, optimization and runtime issues. Some categories in the question paper are pedagogical labels rather than separate formal compiler phases. In particular, “intermediate code generation issue” and “optimization issue” are best understood as problems visible when constructing or improving an intermediate representation, not necessarily errors that make the C source invalid.

(a) Identify Errors

Line	Statement	Type	Reason	Correction
198	return a / b	Syntax	Missing semicolon after return statement.	return a / b;
202	int #value = 20;	Lexical / syntax	# is not permitted inside an identifier and this declaration cannot be tokenized as the intended identifier.	int value = 20;
203	float num = 15.5	Syntax	Missing semicolon.	float num = 15.5;
206	result = divide(10);	Semantic	divide is declared with two parameters, but the call supplies only one.	result = divide(10, 2);
208	if(result = 5)	Logical / semantic warning	= assigns 5 to result; it does not compare result with 5. The expression value 5 is true, so the branch is taken.	if (result == 5)
209	printf("Result is 5\n")	Syntax	Missing semicolon.	printf("Result is 5\n");
214	int z = x / y;	Runtime / undefined behavior	y is initialized to zero, so integer division by zero occurs when	int z = 0; or guard y != 0

			evaluated.	
219	arr[4]	Runtime / bounds	arr[4] is outside the valid range 0–3 for int arr[4].	arr[3] or another valid index
221–222	int *ptr; *ptr = 25;	Runtime / undefined behavior	ptr is uninitialized and is dereferenced without pointing to a valid object.	int value; int *ptr=&value; *ptr=25;
225	x + y * result + 0	Optimization / redundancy	+ 0 is redundant. Multiplication has higher precedence than addition, so the expression is parsed as x + (y * result) + 0.	x + y * result
227–230	while(x < 20) { printf(..., x); }	Logical / runtime	x is never changed in the loop, so if x starts below 20 the condition remains true and the loop does not terminate.	while (x < 20) { printf("%d\n", x); x++; }

Additional semantic note: The declaration int value = 20; is unused in the supplied program after correcting #value. This is not an error that prevents compilation, but a compiler may warn about an unused variable. Also, because the source contains earlier syntax/lexical errors, a real compiler may not reach or reliably diagnose every later issue until those errors are fixed.

(b) Classify by Compiler Phase

Category requested	**Q3 example**	**Phase / interpretation**
Lexical	#value	Lexical scanning encounters # in a position where it cannot form the intended identifier.
Syntax	return a / b; missing ';'; float declaration missing ';'; printf missing ';'	Parsing requires complete statements.
Semantic	divide(10)	The function interface requires two arguments; visible declaration

		permits semantic checking.
Intermediate-code / precedence	x + y * result + 0	The IR should preserve multiplication precedence: t1 = y * result; t2 = x + t1; t3 = t2 + 0. A correct optimizer can simplify t3 to t2.
Optimization	+ 0	Identity operation is redundant when integer/real arithmetic semantics permit the simplification. Removing it reduces unnecessary IR instructions.
Runtime	x/y with y=0; arr[4]; *ptr	These operations can be syntactically and semantically valid yet unsafe when the actual runtime values make the operation invalid.

Operator precedence is not itself an error in x + y * result + 0: standard C gives * higher precedence than +, so y * result is evaluated as a subexpression before the additions. The actual improvement is to generate and then simplify the intermediate representation without changing that required order.

(c) Compilation versus Execution

- Lexical and syntax errors normally prevent successful translation to executable code. A compiler may continue scanning/parsing for diagnostics, but it cannot produce a correct executable from invalid source.
- The divide(10) call is a compile-time semantic/interface problem when the function prototype is visible. The compiler can diagnose the wrong number of arguments before execution.
- The assignment in if(result = 5) is legal C syntax and type-correct for int. It is therefore not necessarily a compile-time error. A good compiler may warn because assignment used as a condition is suspicious.
- Division by zero with y initialized to zero may remain until execution because the compiler does not have to treat every such case as a required compile-time error. Static analysis may nevertheless warn or diagnose a definite case.
- arr[4] is syntactically valid and may compile. Without sufficient range analysis, the out-of-bounds access is detected only when executed or by runtime instrumentation.
- *ptr with an uninitialized pointer can also compile because the declaration is legal. The undefined behavior occurs when the invalid pointer is dereferenced.

- The loop can compile normally but execute indefinitely if x remains less than 20. This is a logical error rather than a grammar error.
- Early errors can prevent later analysis because the parser may lose synchronization. A missing semicolon or malformed token can cause subsequent source text to be interpreted incorrectly, producing cascading diagnostics.

(d) Correct the Program

```
#include <stdio.h>

int divide(int a, int b) {
    return a / b;
}

int main() {
    int value = 20;
    float num = 15.5;
    int result;

    result = divide(10, 2);

    if (result == 5) {
        printf("Result is 5\n");
    }

    int x = 10;
    int y = 0;
    int z = 0;

    if (y != 0) {
```

```
        z = x / y;
    } else {
        printf("Division by zero is not allowed\n");
    }

    int arr[4] = {1, 2, 3, 4};
    printf("%d\n", arr[3]);

    int data = 25;
    int *ptr = &data;
    *ptr = 25;

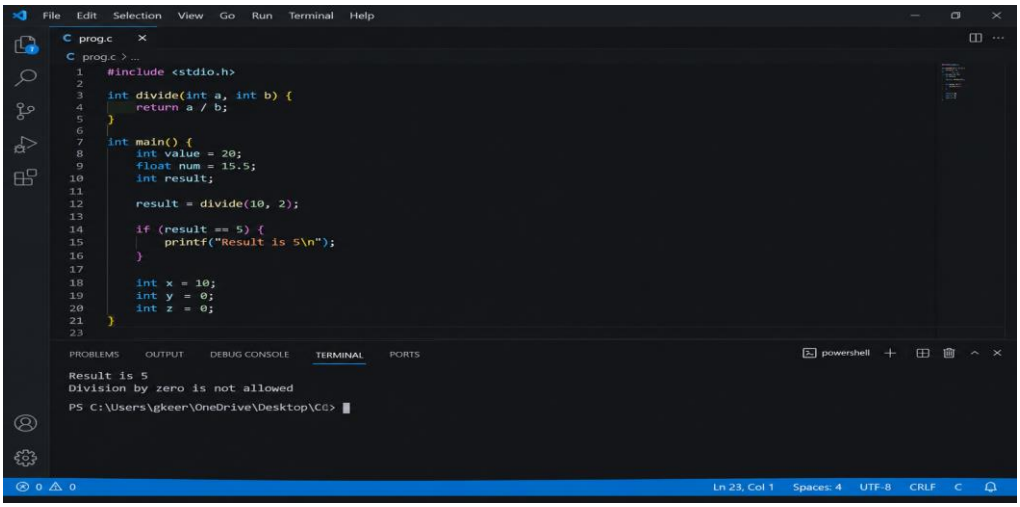
    int total;
    total = x + y * result;

    while (x < 20) {
        printf("%d\n", x);
        x++;
    }

    printf("Value = %d\n", value);
    printf("Number = %.1f\n", num);
    printf("Division Result = %d\n", result);
    printf("Total = %d\n", total);
    printf("Pointer Value = %d\n", *ptr);

    return 0;
}
```

OUTPUT SCREENSHOT:



This corrected version removes the invalid # identifier, restores missing semicolons, supplies both divide arguments, changes assignment to comparison, avoids division by zero, uses a valid array index, initializes the pointer before dereferencing it, removes the redundant + 0, and increments x so the loop terminates.

(e) Deep Analysis

I. Pointer usage: A pointer stores an address. Declaring int *ptr; does not automatically make ptr point to an int object. Its value is indeterminate until initialized. Dereferencing such a pointer attempts to access memory through an invalid/unknown address and therefore produces undefined behavior. Initializing ptr with &value establishes a valid target object for the lifetime of the access.

II. Array bounds: For int arr[4], the valid indices are 0, 1, 2 and 3. arr[4] refers one element beyond the array. C does not automatically perform bounds checking. The generated address calculation can therefore access memory belonging to another object or inaccessible memory. The effects may include corrupted data, incorrect results, crashes or security vulnerabilities.

III. Operator precedence and intermediate code: The expression x + y * result + 0 is parsed according to C precedence as (x + (y * result)) + 0. A compiler may construct an IR such as t1 = y * result; t2 = x + t1; t3 = t2 + 0. Since adding zero does not change the mathematical value under the relevant integer semantics, an optimizer can replace t3 with t2. The key point is that optimization must preserve the language-defined evaluation semantics; it must not rearrange expressions merely because a different order looks simpler.

(f) Advanced Task

I. Two improvements in compiler design:

- Static analysis and sanitizer-assisted diagnostics: use constant propagation, range analysis, definite-initialization analysis and control-flow analysis during compilation. For example, constant propagation can reveal y = 0 before x / y; range analysis can flag arr[4] when the array bound is known; initialization

analysis can warn about an uninitialized pointer. During testing, sanitizers can instrument the executable to report actual invalid accesses with source locations.

- Source-aware diagnostic infrastructure: attach line/column information and original-source metadata to tokens and intermediate-representation instructions. Diagnostics can then show the offending expression, the expected form, the reason, and a suggested fix. Example: “Line 206: function ‘divide’ expects 2 arguments but 1 was supplied; add a second integer argument.”

II. Removing redundant expressions during optimization: Build an abstract syntax tree and/or intermediate representation, then apply algebraic simplification rules such as $x + 0 \rightarrow x$, $x * 1 \rightarrow x$, and constant folding such as $3 * 4 \rightarrow 12$ when the transformation is valid under C semantics. Dead-code elimination can remove computations whose results are never used, while common-subexpression elimination can reuse an already computed expression. Each transformation must preserve observable behavior, including side effects, overflow rules and floating-point constraints where relevant.

Cross-Question Technical Concepts

Lexical Analysis and Token Formation

Lexical analysis converts a character stream into tokens such as keywords, identifiers, constants, string literals, character constants, operators and punctuation. A scanner usually implements regular-language patterns using finite-state-machine ideas. For example, an identifier can be described informally as: first character = letter or underscore; remaining characters = letters, digits or underscores. A numeric constant follows separate rules for decimal, octal, hexadecimal and floating forms.

Syntax versus Semantic Errors

Syntax errors violate the grammar of the language, such as a missing semicolon or unmatched delimiter. Semantic errors occur when the token sequence is grammatically valid but violates language meaning or type/interface constraints, such as calling a declared two-parameter function with one argument. Logical errors produce a program that can compile but does not implement the intended behavior, such as a non-terminating loop.

Runtime Safety

Runtime errors arise from values and execution paths that cannot necessarily be determined from syntax alone. Division by zero, invalid pointer dereferencing and out-of-bounds array access are major examples in these programs. They are especially important in C because the language provides little automatic memory or bounds protection.

Assignment versus Equality

In C, '=' is assignment and '==' is equality comparison. Thus if (result = 5) assigns 5 to result and then uses the assigned value as the condition. Since 5 is nonzero, the condition is true. The intended comparison is if (result == 5). Compiler warnings such as “suggest parentheses around assignment used as truth value” can help detect this mistake.

Function Declarations and Argument Matching

A function declaration/prototype communicates the parameter types and count to the compiler. With int divide(int a, int b), a call should supply two compatible arguments. A call with one argument is inconsistent with the visible prototype and should be diagnosed. Correct declarations also enable stronger type checking and better diagnostics.

Compiler Error Recovery

Compilers attempt to recover after errors so that multiple diagnostics can be reported. Lexical recovery may skip invalid characters or consume input to a delimiter. Parser recovery often synchronizes at semicolons, braces or statement boundaries. Recovery must balance continuing analysis with avoiding misleading cascades.

Compilation versus Execution

Compilation-time detection is strongest for lexical structure, grammar and many type/interface constraints. Execution-time checking is required for many value-dependent memory and arithmetic conditions. Static analysis bridges the gap by reasoning about possible runtime states before execution. Sanitizers then provide dynamic checking during testing.

Conclusion

The three programs demonstrate why a compiler must analyze source code through multiple complementary stages. Lexical analysis recognizes valid tokens and identifies malformed identifiers, literals and symbols. Syntax analysis checks whether those tokens form valid language constructs. Semantic analysis checks meaning, declarations, types and function interfaces. Intermediate representation preserves language-defined precedence and evaluation structure, while optimization removes safe redundancies such as an unnecessary + 0 without changing observable behavior. Finally, runtime safety mechanisms are essential for value-dependent problems such as division by zero, invalid pointer dereferences, array bounds violations and non-terminating loops. Effective compilers therefore combine accurate diagnostics, recovery, static analysis, optimization and runtime instrumentation to make programs safer, easier to debug and more efficient.

Source cross-check: The answer structure follows the supplied assessment's Q1(a–f), Q2(a–e), and Q3(a–f) tasks, including the requested error identification, classification, compilation/execution analysis, correction, impact/deep analysis, and advanced compiler-design questions.